Practical 2

Aim : Study and implement about Bitcoin Structure and Design.

Theory:

Bitcoin is a decentralized digital currency that uses blockchain technology to record transactions without depending on a central authority. Transactions are grouped into blocks, and these blocks are linked using cryptographic hashes.

Key structural elements of Bitcoin:

- Block – Contains transaction data, a timestamp, a reference to the previous block's hash, and a nonce used during Proof of Work.
- Blockchain – A sequence of blocks linked through cryptographic hashes. Changing a block changes its hash and can break the chain.
- Hashing (SHA-256) – Bitcoin uses SHA-256 to generate fixed-length cryptographic hashes that help maintain data integrity.
- Proof of Work (Mining) – Miners repeatedly change the nonce until a block hash satisfies a specified difficulty condition, such as beginning with a number of zeros.
- Merkle Tree – Transactions are hashed and combined pairwise to produce a Merkle root. It is part of Bitcoin's block structure but is not implemented in this simplified simulation.
- Decentralization – Copies of the blockchain are maintained by many nodes, and Bitcoin's consensus mechanism helps nodes agree on the valid blockchain.

Implementation – Simplified Blockchain Simulation in Python

```
import hashlib
import json
import time
class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()
    def calculate_hash(self):
        content = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
```

```
            "data": self.data,
            "previous_hash": self.previous_hash
        }, sort_keys=True)
        return hashlib.sha256(
            content.encode()
        ).hexdigest()

genesis = Block(0, "Genesis Block", "0")
block1 = Block(1, "Alice pays Bob", genesis.hash)
block2 = Block(2, "Bob pays Carol", block1.hash)
for block in [genesis, block1, block2]:
    print("Block:", block.index)
    print("Data:", block.data)
    print("Previous Hash:", block.previous_hash)
    print("Current Hash:", block.hash)
    print()
```

Output:

```
PS D:\ADIT_DESKTOP\SEM - 7th\BCHAIN> python practical_2.py
Block: 0
Data: Genesis Block
Previous Hash: 0
Current Hash: fb4f1ae26e6c0c1d47ec58a4664a5eaf7e70fa1d1dee45a545cab5f7c554ad0e

Block: 1
Data: Alice pays Bob
Previous Hash: fb4f1ae26e6c0c1d47ec58a4664a5eaf7e70fa1d1dee45a545cab5f7c554ad0e
Current Hash: e78efc7ff9c30572f01caaf50620d9092b92847c582b3893e1e3bac473a4ba6d

Block: 2
Data: Bob pays Carol
Previous Hash: e78efc7ff9c30572f01caaf50620d9092b92847c582b3893e1e3bac473a4ba6d
Current Hash: 1a9694c27abc208258b0cf2df7c06a731676f4283b65e4cd28284644c756a75d
```